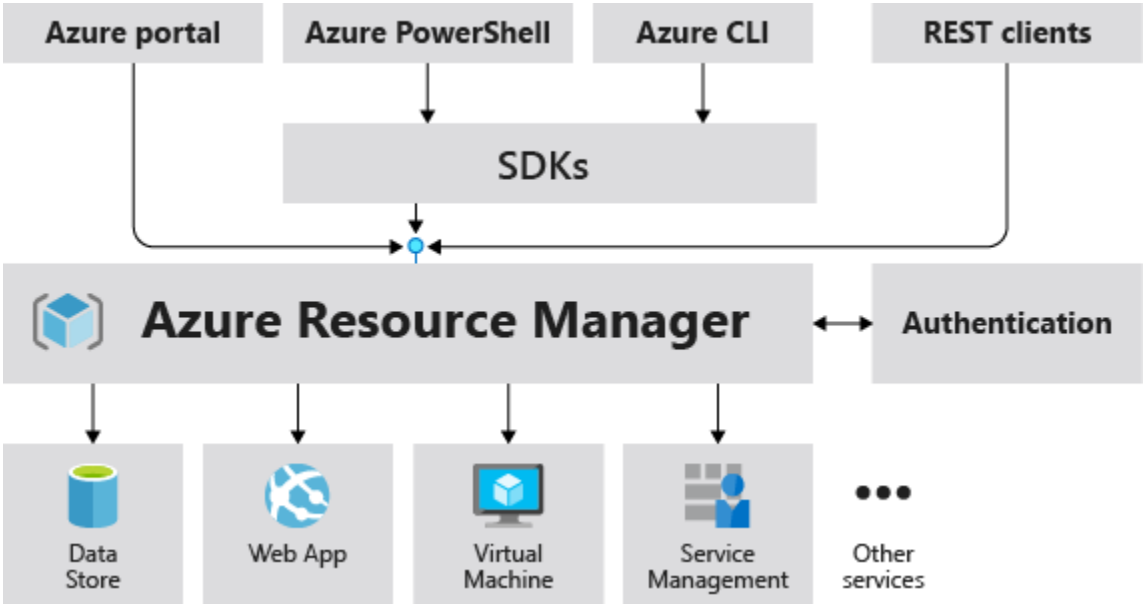
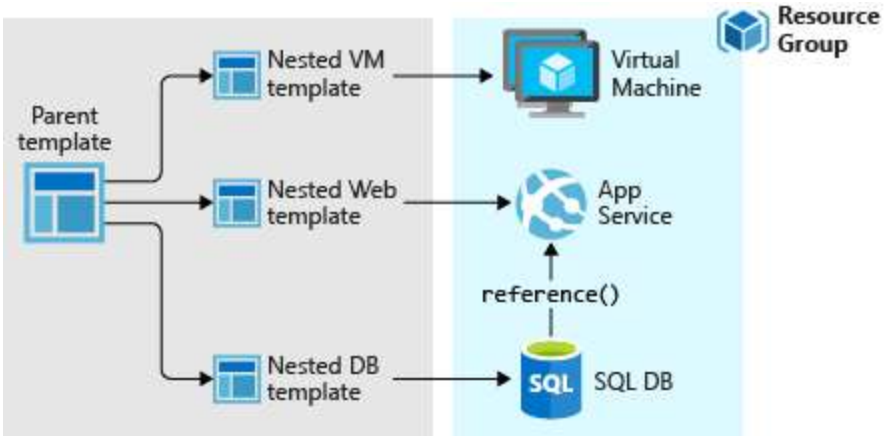
ARM Templates in Azure

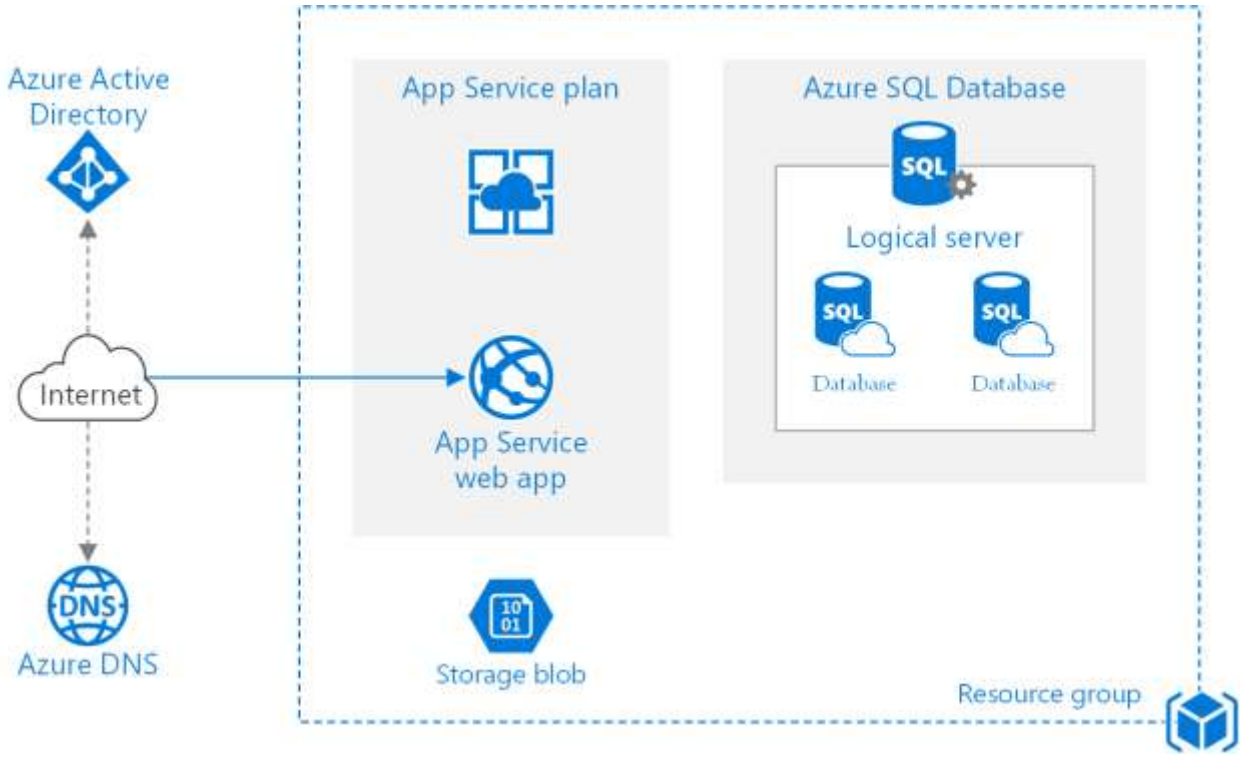
What is an ARM Template in Azure?

An **ARM Template** (Azure Resource Manager Template) is a **JSON-based Infrastructure as Code (IaC)** file used to **define, deploy, and manage Azure resources** in a **declarative** manner.

Instead of creating resources manually through the Azure Portal, you describe **what resources you want** and **Azure handles how to create them**.







Why ARM Templates Are Needed

Manual resource creation leads to:

- Configuration drift
- Human errors
- Inconsistent environments (Dev / Test / Prod)

ARM Templates solve this by enabling:

- **Repeatable deployments**
- **Version control**
- **Automated infrastructure provisioning**
- **Environment consistency**

Key Characteristics of ARM Templates

1. Declarative Syntax

You define **what** you want, not **how** to do it.

Example:

“Create one App Service, one SQL Database, and one Storage Account”

Azure decides the correct execution order.

---

2. Idempotent

You can deploy the **same template multiple times**, and Azure ensures:

- No duplicate resources
- Only required changes are applied

---

3. JSON Format

ARM templates are written in **JSON**, making them:

- Machine-readable
- Source-control friendly (Git)
- CI/CD compatible

---

4. Dependency Management

Azure automatically understands resource dependencies using:

"dependsOn": []

Example:

- App Service depends on App Service Plan
- SQL Database depends on SQL Server

Core Components of an ARM Template

An ARM template typically contains the following sections:

Section	Purpose
$schema	Defines ARM template version
contentVersion	Template versioning
parameters	Inputs provided at deployment time
variables	Reusable values
resources	Azure resources to deploy
outputs	Values returned after deployment

Simple ARM Template Example

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "storageAccountName": {
      "type": "string"
    }
  },
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2022-09-01",
      "name": "[parameters('storageAccountName')]",
      "location": "East US",
      "sku": {
```

```
      "name": "Standard_LRS"
    },
    "kind": "StorageV2"
  }
 ]
}
```

This template:

- Takes a storage account name as input
- Deploys a Storage Account in Azure

How ARM Template Deployment Works

1. Template is submitted (Portal / CLI / PowerShell / Pipeline)
2. Azure Resource Manager:
    - Validates template
    - Resolves dependencies
    - Deploys resources in correct order
3. Deployment result is returned

Ways to Deploy ARM Templates

You can deploy ARM templates using:

- **Azure Portal**
- **Azure CLI**

- **PowerShell**
- **Azure DevOps Pipelines**
- **GitHub Actions**

Example (CLI):

```
az deployment group create \
  --resource-group MyRG \
  --template-file template.json \
  --parameters storageAccountName=mystorage123
```

ARM Template vs Other IaC Tools

Tool	Type
ARM Template	Native Azure IaC
Bicep	Simplified ARM (recommended)
Terraform	Multi-cloud IaC
Pulumi	Code-based IaC

Note: **Bicep** compiles into ARM Templates and is now Microsoft's preferred approach.

Real-Time Use Cases

ARM templates are commonly used for:

- Dev / Test / Prod environment setup
- CI/CD pipeline infrastructure provisioning
- Disaster recovery automation
- Standardized enterprise deployments
- Compliance and governance enforcement

Advantages of ARM Templates

- Native Azure support
- No external tools required
- Strong role-based access control (RBAC)
- Deep Azure service integration
- Reliable and production-ready

Limitations

- JSON can become verbose
- Harder to read for beginners
- Less flexible than programming languages

(Reason why **Bicep** exists)
